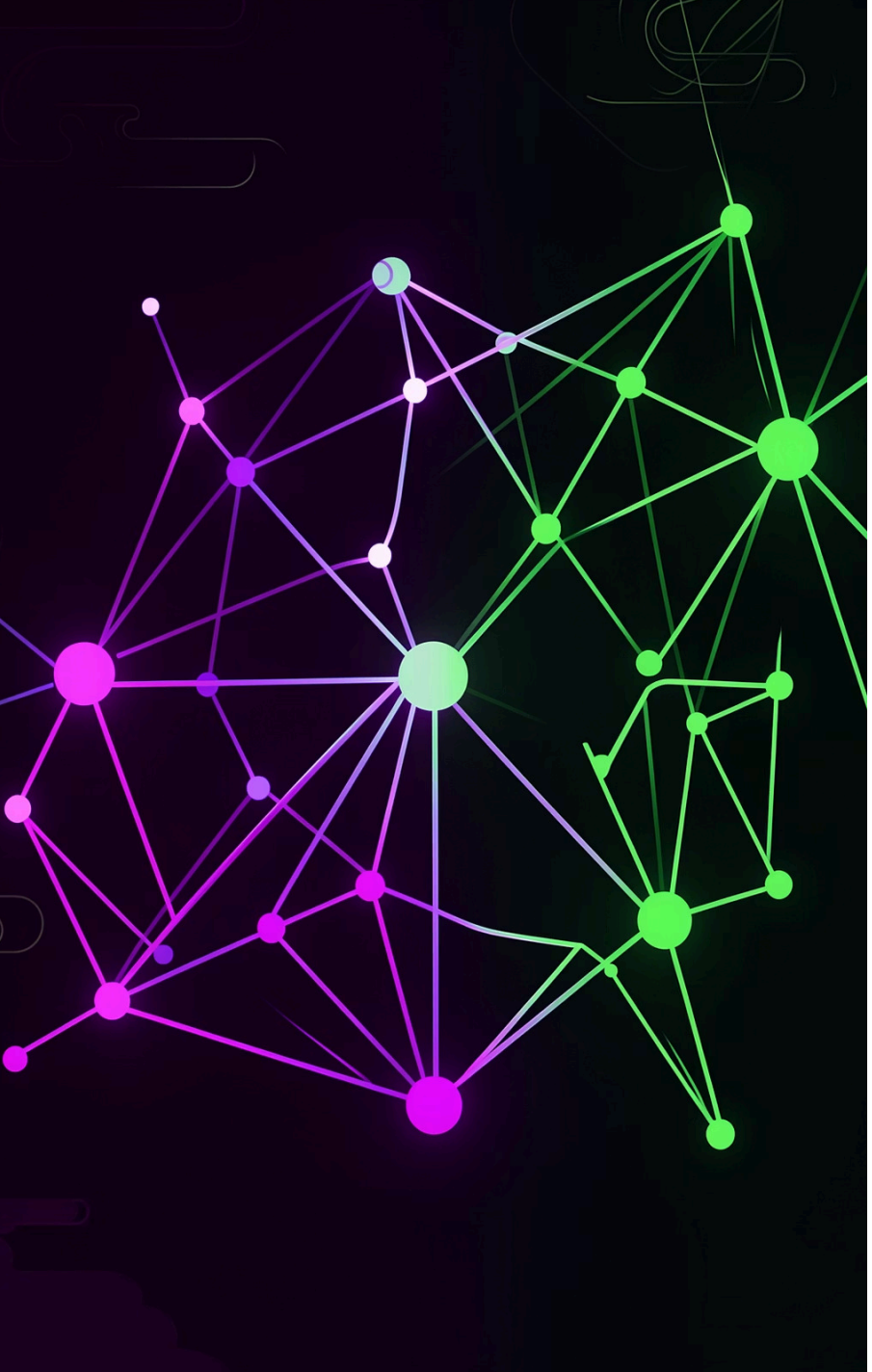


TRADEN-PROD

Atomic Fee Enforcement in Decentralized Perpetual Trading on Solana

By prepending a SystemProgram.transfer fee instruction into the same Solana transaction message that carries a Drift trade, I make the user's Ed25519 signature mathematically bind both — fee and trade either land together or revert together, with no smart contract.

Made with GAMMA

Today's DEX Fee Models Are Trust-Based — and Trivially Bypassable

❌ Broken Status Quo

Two separate transactions: [User Order] → [TX #1: Trade] ✅ lands → [TX #2: Fee] (separate) ❌ user can drop, censor, front-run, or simply not sign the second tx

Two separate transactions = two separate signatures = two separate failure modes.

✅ My Model

One atomic transaction: [User Order] → [SINGLE TX: Fee instruction + Trade instruction] → [ONE Ed25519 signature covers BOTH] → Solana runtime executes atomically → both succeed OR both revert

One transaction · one signature · zero trust.

Why two-transaction designs fail

The second tx is independently droppable. The user's wallet can refuse to sign it, an MEV searcher can front-run it, a malicious relayer can censor it — and the trade has already executed.

Why on-chain fee programs are weak

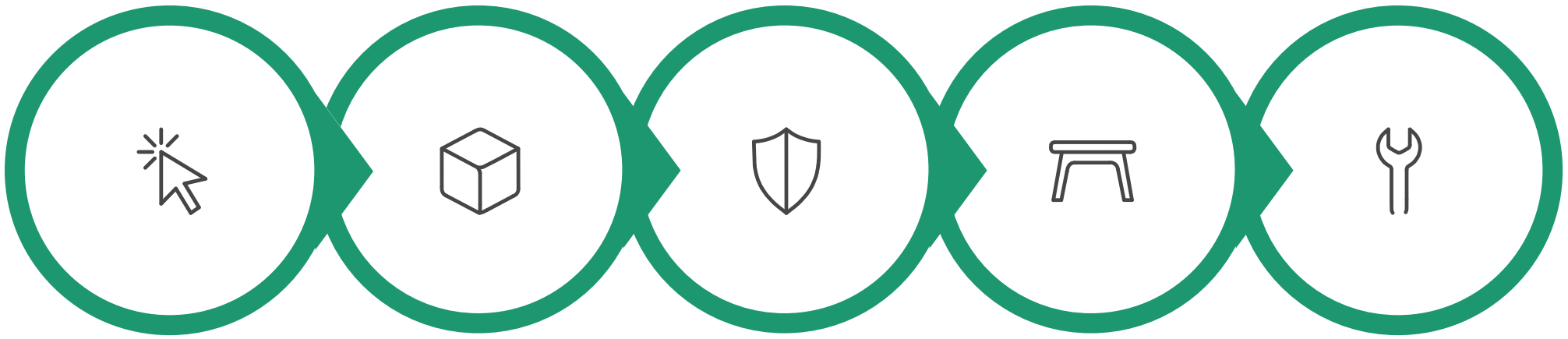
They require a custom deployed program (audit cost, lock-in, upgrade risk) and the user's client can still skip the wrapper and call Drift directly.

Why my approach is different

The fee is not "added by the platform after" — it is part of the message the user signs. The user's wallet cannot un-sign one half of a signed message.

Made with GAMMA

How a Single Ed25519 Signature Becomes the Enforcement Mechanism



Place Order

Build V0

Intercept Send

Resolve ALTs

Decompile & Prepend

Ed25519 signatures cover the entire message, byte-for-byte

Flip one bit anywhere — including the fee amount, the recipient pubkey, or the order of instructions — and SHA-256's avalanche property guarantees a different digest, which the validator's verifier rejects.

Solana's runtime executes atomically

The instruction list of a transaction is executed inside a single account-state commit. There is no scheduling point between instructions where one can land without the other.

The fee becomes part of the signed hash

Removing it changes the hash; modifying it changes the hash; reordering it changes the hash. Every adversary action invalidates the signature.

Made with GAMMA

A Five-Layer Architecture, with the Contribution Concentrated in Layer 3

1	**1. Presentation** Next.js 15 · React 19 · Tailwind · shadcn/ui 13 routes: /perps /spot /signals /user /admin /creator /receipt /blockchain /verify /security /benchmarks /explorer
2	**2. State & Sync** Zustand stores + React Query + AuthorityDrift's WebSocket subscriptions for oracle / mark-price / user-account streams
3	**3. ⭐ Atomic Fee Enforcement** DriftClientWrapper.ts intercepts sendTransaction, decompiles the V0 message, prepends a fee transfer, recompiles, forwards. tradingFee.ts computes the 5-bps amount and converts USDC→SOL via Pyth.
4	**4. Protocol & Wallet** Drift SDK · @solana/web3.js · Wallet Adapter · tweetnacl (browser-side Ed25519 / SHA-256 demos)
5	**5. Persistence & APIs** Next.js route handlers + MongoDB (audit trail) + Pyth & Switchboard oracle feeds

Layer 1 is split deliberately: 7 routes serve the trading product; 6 routes serve as a verification surface that lets anyone confirm the cryptographic claims live. Layer 3 is the entire contribution. Everything above and below it is conventional Solana/Next.js engineering. The interceptor is where the cryptographic guarantee gets created.

Made with GAMMA

The Five Modules That Make the Contribution Real

Core Libraries (src/lib/)

DriftClientWrapper.ts

Wraps drift.driftClient.sendTransaction. For a V0 message: resolves every Address Lookup Table, decompiles to a TransactionMessage, prepends a SystemProgram.transfer instruction, recompiles with the same payer / blockhash / ALTs, and forwards.

tradingFee.ts

Computes feeLamports = orderSize × 5/10000. For USDC-quoted markets, reads the live SOL/USDC price from Drift's oracle cache (Pyth + Switchboard) and converts the USDC fee to lamports before injection.

cpi.ts

Pure-TS extractor: turns a ParsedTransactionWithMeta's flat meta.innerInstructions list into a tree {parent → CPI children}, with friendly labels for System / Drift / SPL Token / Pyth / Switchboard.

merkle.ts

SHA-256 Merkle tree from scratch. Domain-separated leaf (0x00 ‖ x) and node (0x01 ‖ L ‖ R) hashing to defeat second-preimage attacks. Exposes buildMerkleTree, getProof, verifyProof.

solscan.ts

Env-aware Solscan URL helpers (devnet vs mainnet). Used everywhere a signature is rendered so any reviewer can independently verify.

New Routes I Added

/blockchain

Concept-to-source map of every primitive used

/verify

4 in-browser cryptographic demos (no wallet)

/security

6 attack simulations (live, with byte-level diffs)

/benchmarks

tx-size / signing-latency / CU overhead numbers

/explorer

Paste any signature → render the live CPI tree

/receipt/[s]

Shareable per-trade receipt, links to Solscan

6 new lib modules · 6 new routes · 6 API routes hardened · ~9 k LoC delta

Made with GAMMA

A Working Trading Platform With a Built-In Verification Surface

Trading Platform

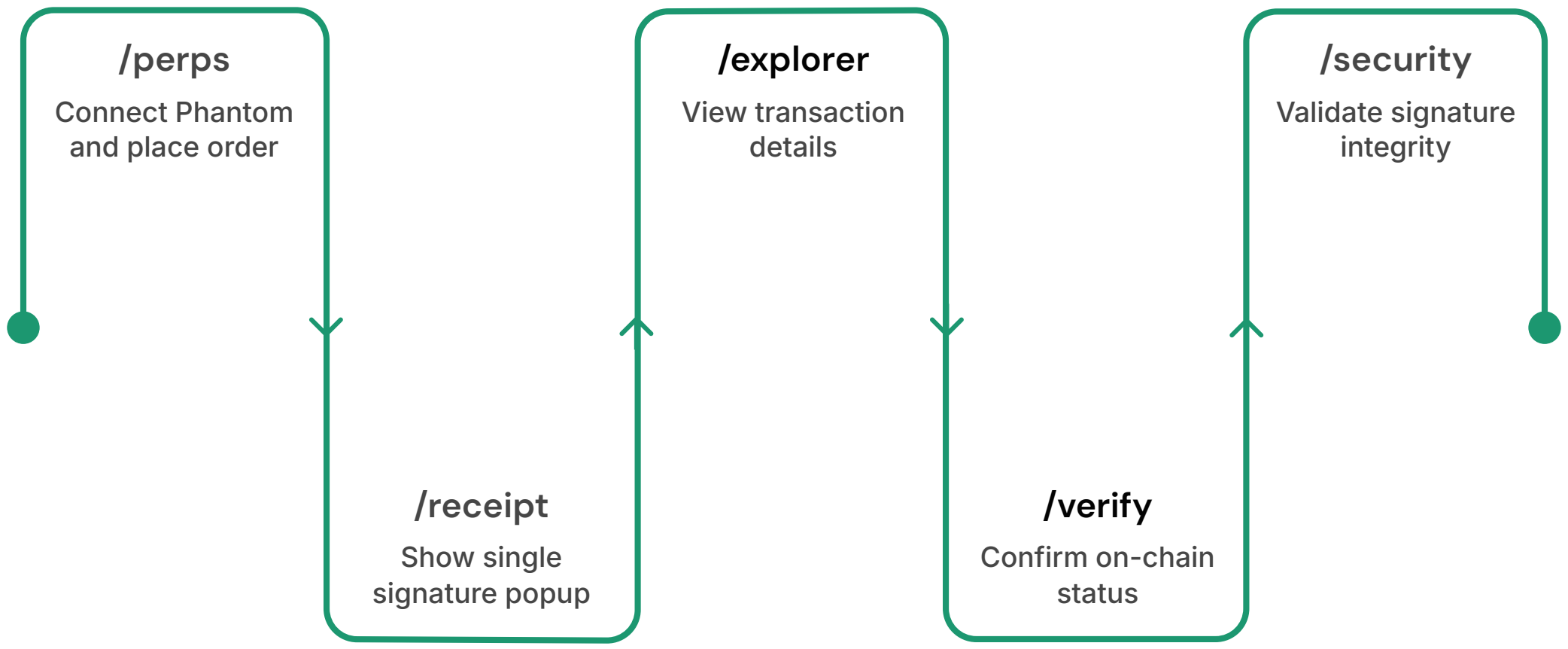
Real perpetual trading via Drift, integrated end-to-end: 40+ perpetual markets with live oracle + mark prices; Lightweight-charts candlesticks + WebSocket orderbook; 5 order types (market, limit, take-profit, stop-loss, oracle-limit); Spot deposit / withdraw / swap (USDC, SOL, etc.); Phantom + Solflare wallet integration; Trading-signal marketplace with copy-execution; Creator revenue sharing (50/50 platform vs creator); Admin & creator dashboards with Solscan deep-links.

Verification Surface

Six routes that turn every claim into something a reviewer can click and replay: /blockchain — every primitive mapped to a file in the repo; /verify — 10-step Ed25519+SHA-256 wizard, hash avalanche visualisation, Merkle proof + tamper test; /security — replay, signature forgery, fee strip, fee-amount tamper, instruction reorder, MITM recipient swap; /benchmarks — bytes / latency / compute-units measured live; /explorer — full CPI tree from meta.innerInstructions; /receipt — per-tx proof card with the signed digest.

Made with GAMMA

Live Demo — A 5-Minute Walkthrough That Proves the Thesis End-to-End



01

/perps

Click: Connect Phantom (devnet) → SOL-PERP → place a 0.01-SOL market order. **Point:** Phantom popup — show that ONE signature is requested. **Say:** "One click here triggers the bundle. The wallet is being asked to sign the fee + trade together, not separately."

02

/receipt/

Click: Open the receipt route for the signature that just landed. **Point:** Status: confirmed. The fee row + trade row both reference the same signature. **Say:** "Same signature, both sides. That's the on-chain proof."

03

/explorer

Click: Paste the same signature → Verify. **Point:** The three green pills (fee found / trade found / both in same atomic tx). Then the Instruction & CPI tree — SystemProgram.transfer at depth 0 sitting next to the Drift instruction, with Drift's CPIs (System, SPL Token, Pyth, Switchboard) indented under it. **Say:** "This tree is read straight from meta.innerInstructions on chain. The fee instruction is physically inside the same transaction envelope as the trade — anyone can verify it."

04

/verify

Click: Run Section 1's 10-step integrity wizard. Then scroll to Section 4 (Merkle proofs) and tamper a leaf. **Point:** Step 9 — the tampered transaction's hash differs in ~50 % of bits (avalanche). Step 10 — the original signature is now INVALID. **Say:** "These run client-side, no wallet needed. A 1-lamport edit to the fee invalidates the Ed25519 signature. Tampering a Merkle leaf invalidates its proof against the published root."

05

/security

Click: Run all attack tests. **Point:** All six cards turn green; expand any card to see the byte-level diff and the verdict. **Say:** "Six classical attacks, all defeated by construction: replay, signature forgery, fee strip, fee-amount tamper, instruction reorder, MITM recipient swap."

06

/benchmarks → /blockchain

Click: Run all benchmarks. Then jump to /blockchain. **Point:** +64 bytes overhead, < 1 ms extra latency, ~150 / 200 000 CU, ~0.07 % total relative overhead. On /blockchain, expand any concept card — every one links to a real file. **Say:** "Overhead is well under 1 % everywhere. And every primitive I claimed is wired to an actual file in the repo."

ⓘ If devnet is unreliable, /verify and /security alone fully prove the thesis — they need no wallet, no RPC, no server.

Made with GAMMA

Six Classes of Attacks I Implemented as Live Tests — All Defeated

Attack	Why it fails	How my test exercises it	Status
Fee-instruction removal	msg hash changes → Ed25519 verify fails	build [fee, trade]; rebuild without ix[0]; verify old sig vs new msg	✅
Fee-amount manipulation	SHA-256 avalanche flips ≈ 50 % of digest	flip 1 lamport in fee; count differing bits in the two SHA-256 outputs	✅
Recipient swap (MITM)	account keys are part of the signed message	replace recipient pubkey with attacker pubkey; verify old sig — fails	✅
Replay attack	recentBlockhash window is also signed; expires after 150 blocks (60s)	sign with blockhash A; rebroadcast after blockhash B is current	✅
Signature forgery	Ed25519 = 128-bit ECDLP on Curve25519	sign with key A; verify with key B; flip a byte in sig and re-verify	✅
Instruction reordering	message bytes change → different SHA-256 digest	swap ix[0] and ix[1]; verify old sig — fails	✅

Original SHA-256:
a3f2c1e4b5d6...7890

Tampered SHA-256:
x9p8q7r6s5t4...1234

1-lamport change → 128-ish bit flip (avalanche effect)

Made with GAMMA

~0.07 % Overhead Across Every Metric I Could Think to Measure

+64

Bytes

Tx-size overhead per fee instruction (serialized V0 msg)

<1

Milliseconds

Extra signing latency (100-iter avg in the browser)

~150

Compute Units

CU cost of one SystemProgram transfer instruction / 200 000 CU

~0.07

Percent

Total relative overhead vs the unbundled trade

Every number above is measured live in the browser, not estimated. The reason it is so cheap is that I do not deploy any new on-chain code — I only add one System Program transfer instruction to a transaction that was going to land anyway.

Blockchain Primitives This Project Exercises

Asymmetric Crypto

Ed25519 / Curve25519 — One Ed25519 sig used by every wallet

SHA-256 + Merkle

merkle.ts + /verify §3–4 — sig covers all ix

Wallets & Addresses

Phantom · base58 keys — Drift in 1 tx

Transactions

V0 messages + Address Lookup Tables

Atomic Execution

SystemProgram + cpi.ts + /explorer tree

Replay Protection

recentBlockhash window (~150 blocks / ~60 s)

Decentralised Oracles

Pyth + Switchboard drive USDC→SOL conversion

Programs + CPI

meta.innerInstructions — verified live

Made with GAMMA

Monetisation Without Trust. Trading Without Compromise.

Cryptographic Guarantee

Ed25519 + SHA-256 + Solana's atomic execution compose into an unbypassable platform-fee primitive — no new on-chain code required.

Zero On-Chain Footprint

Nothing new is deployed. The contribution lives entirely in client-side TypeScript. Audit cost: zero. Lock-in: zero. Portable to any Solana DEX SDK.

Empirically Verified

Six attacks implemented as live tests, all defeated. Four overhead metrics measured in the browser. Every claim links to a clickable demo in the deployed app.

What's Next

Mainnet Deployment

Production builder

Cross-chain Port

Sui / Aptos / Sei

Dynamic Fee Engine

DAO-governed authority + signed tier-aware fees

Monetisation Without Trust

Trading Without Compromise

Viyom Gupta · 2023A7PS0413G · github.com/Viyom10/traden

Made with GAMMA